Stat411PipeliningAndAnalysis

December 10, 2024

1 Statistical Modeling of the Old School Runescape Highscores

2 Pipelining and Analysis

2.1 Author: Caleb Hart

2.2 Oregon Institutute of Technology

2.3 Statistical Machine Learning I - Fall 2024

2.4 Date Created: 12/10/2024

Research Question: Can you predict a player's total xp based on the "kill" count of each in-game boss, despite the extremely sparse data? Goal: Predict 'total_xp' based on 'kill count' from in-game boss.

Method will need to be robust to sparse and skewed data. Random Forest Regressor with Linear Regression for comparison

Standard Scaler and Simple Imputation to exasturbate the missing the data

`strategy = 'constant', fill_value = 0`

Kbest features: 5, 10, or 15

Wide Grid Search. Not tuned!

Scoring Metric: Negative MSE

3 Imports

[60]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

[62]:
```
data = pd.read_csv('OSRSkcDataset.csv')
```

I am choosing to drop the username column to avoid the hassle, despite the fact that it has a lot of potential for feature extraction. Since Old School Runescape is an old game(only 11 years now) and there are a limited amount of usernames. Short usernames, the minimum is 3 characters, are thus quite rare and old.

[65]:
```
data = data.drop(columns=['username'], errors='ignore')
```

[67]:
```
# Split the data into features (X) and target (y)
X = data.drop('total_xp', axis=1)  # Features
y = data['total_xp']               # Target

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)
```

[69]:
```
# Create a ColumnTransformer for preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='constant', fill_value=0)),  #␣
 ↪Impute missing values as 0
            ('scaler', StandardScaler())  # Scale features
        ]), X.columns)  # Assuming all columns are numerical
    ]
)
```

[71]:
```
# Create a RandomForest model
rf_model = RandomForestRegressor(random_state=42)
```

[73]:
```
# Create a pipeline for RandomForest that first applies preprocessing, then␣
 ↪fits the model
rf_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('select_k_best', SelectKBest(f_regression)),  # Feature selection
    ('rf', rf_model)
])
```

[75]:
```
# Define the hyperparameter grid for GridSearchCV
param_grid_rf = {
    'select_k_best__k': [5, 10, 15],  # Try selecting 5 or 10 features
    'rf__n_estimators': [50, 100, 200],  # Number of trees
    'rf__max_depth': [3, 5, 10],  # Max depth of trees
    'rf__min_samples_split': [2, 5],  # Minimum samples to split a node
```

```
    'rf__min_samples_leaf': [1, 2],    # Minimum samples per leaf
    'rf__bootstrap': [True, False],    # Bootstrap sampling
}
```

[77]:
```
grid_search_rf = GridSearchCV(rf_pipeline, param_grid=param_grid_rf, cv=3,
                              scoring='neg_mean_squared_error', n_jobs=-1,␣
 ↪verbose=2)
```

[79]:
```
# Fit GridSearchCV to find the best parameters for RandomForest
grid_search_rf.fit(X_train, y_train)
```

```
Fitting 3 folds for each of 216 candidates, totalling 648 fits

C:\Users\caleb\anaconda3\Lib\site-packages\numpy\ma\core.py:2820:
RuntimeWarning: invalid value encountered in cast
  _data = np.array(data, dtype=dtype, copy=copy,
```

[79]:
```
GridSearchCV(cv=3,
             estimator=Pipeline(steps=[('preprocessor',
                                        ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
          SimpleImputer(fill_value=0,
                        strategy='constant')),
         ('scaler',
          StandardScaler())]),
Index(['bounty_hunter_hunter_score', 'bounty_hunter_rogue_score',
       'clue_scrolls_all_score', 'clue_scrolls_beginner_score',
       'clue_scrolls_easy_score',…
                                        SelectKBest(score_func=<function
f_regression at 0x000001B9DD43B7E0>)),
                                       ('rf',
RandomForestRegressor(random_state=42))]),
             n_jobs=-1,
             param_grid={'rf__bootstrap': [True, False],
                         'rf__max_depth': [3, 5, 10],
                         'rf__min_samples_leaf': [1, 2],
                         'rf__min_samples_split': [2, 5],
                         'rf__n_estimators': [50, 100, 200],
                         'select_k_best__k': [5, 10, 15]},
             scoring='neg_mean_squared_error', verbose=2)
```

[80]:
```
# Print the best hyperparameters and the corresponding score
print(f"Best hyperparameters for Random Forest: {grid_search_rf.best_params_}")
print(f"Best cross-validation score (neg MSE): {grid_search_rf.best_score_}")
```

```
Best hyperparameters for Random Forest: {'rf__bootstrap': True, 'rf__max_depth':
10, 'rf__min_samples_leaf': 1, 'rf__min_samples_split': 5, 'rf__n_estimators':
200, 'select_k_best__k': 15}
Best cross-validation score (neg MSE): -6170180294752971.0
```

[81]:
```
# Evaluate the best RandomForest model on the test set
best_rf_model = grid_search_rf.best_estimator_
rf_test_score = best_rf_model.score(X_test, y_test)
print(f"Random Forest Test set score (R^2): {rf_test_score}")
```

```
Random Forest Test set score (R^2): 0.7613915635835926
```

[82]:
```
# Create a pipeline for Linear Regression
lr_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('select_k_best', SelectKBest(f_regression, k=10)),  # Using the same␣
 ↪preprocessing as RandomForest
    ('lr', LinearRegression())
])
```

[83]:
```
# Fit the Linear Regression pipeline
lr_pipeline.fit(X_train, y_train)
```

[83]:
```
Pipeline(steps=[('preprocessor',
                 ColumnTransformer(transformers=[('num',
                                                  Pipeline(steps=[('imputer',
SimpleImputer(fill_value=0,
 strategy='constant')),
                                                                  ('scaler',
StandardScaler())]),
Index(['bounty_hunter_hunter_score', 'bounty_hunter_rogue_score',
       'clue_scrolls_all_score', 'clue_scrolls_beginner_score',
       'clue_scrolls_easy_score', 'clue_scrolls_medium_score',…
       'theatre_of_blood_score', 'theatre_of_blood_hard_mode_score',
       'thermonuclear_smoke_devil_score', 'tzkal_zuk_score', 'tztok_jad_score',
       'venenatis_score', 'vetion_score', 'vorkath_score', 'wintertodt_score',
       'zalcano_score', 'zulrah_score'],
      dtype='object'))])),
                ('select_k_best',
                 SelectKBest(score_func=<function f_regression at
0x000001B9DD43B7E0>)),
                ('lr', LinearRegression())])
```

[84]:
```
# Evaluate Linear Regression on the test set
lr_predictions = lr_pipeline.predict(X_test)
lr_mse = mean_squared_error(y_test, lr_predictions)
lr_test_score = lr_pipeline.score(X_test, y_test)
```

[85]:
```
print(f"Linear Regression Test set score (R^2): {lr_test_score}")
print(f"Linear Regression Test set MSE: {lr_mse}")
```

```
Linear Regression Test set score (R^2): 0.47134066067433134
Linear Regression Test set MSE: 3688426587268186.5
```

4 Model Evaluation

4.0.1 Selected Paramters for the Random Forest

[88]:
```
selected_features_rf = X_train.columns[best_rf_model.
 ↪named_steps['select_k_best'].get_support()]
print(f"Selected predictors for Random Forest: {list(selected_features_rf)}")
```

```
Selected predictors for Random Forest: ['clue_scrolls_hard_score',
'clue_scrolls_elite_score', 'clue_scrolls_master_score', 'cerberus_score',
'chambers_of_xeric_score', 'dagannoth_prime_score', 'dagannoth_supreme_score',
'hespori_score', 'kalphite_queen_score', 'kraken_score', 'mimic_score',
'skotizo_score', 'thermonuclear_smoke_devil_score', 'tztok_jad_score',
'wintertodt_score']
```

4.0.2 Selected Parameters for the Linear Regression

[90]:
```
# Get the selected features for Linear Regression
selected_features_lr = X_train.columns[lr_pipeline.named_steps['select_k_best'].
 ↪get_support()]
print(f"Selected predictors for Linear Regression:␣
 ↪{list(selected_features_lr)}")
```

```
Selected predictors for Linear Regression: ['clue_scrolls_hard_score',
'clue_scrolls_elite_score', 'clue_scrolls_master_score',
'dagannoth_prime_score', 'dagannoth_supreme_score', 'kalphite_queen_score',
'kraken_score', 'skotizo_score', 'tztok_jad_score', 'wintertodt_score']
```

[98]:
```
# Make predictions on the test set
y_pred = best_rf_model.predict(X_test)

# Evaluate the model performance
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Print the evaluation metrics
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"Mean Absolute Error (MAE): {mae}")
print(f"R² Score: {r2}")
```

```
Mean Squared Error (MSE): 1664757690552418.8
Root Mean Squared Error (RMSE): 40801442.26068999
Mean Absolute Error (MAE): 24966611.886259284
R² Score: 0.7613915635835926
```

Negative Mean Squared Error (NMSE): -6170180294752971.0

Mean Squared Error (MSE): 1664757690552418.8

Root Mean Squared Error (RMSE): 40801442.26068999

Mean Absolute Error (MAE): 24966611.886259284

R^2 Score: 0.7613915635835926

Cross-Validation Score (Negative MSE): The negative MSE is approximately -6.17e+15. This is a large negative number, but remember that the MSE is the mean squared error, and the smaller the negative value, the better the model fits the data. You can take the absolute value of the negative MSE to interpret it as a regular MSE score, but the key here is the test set R^2 score.

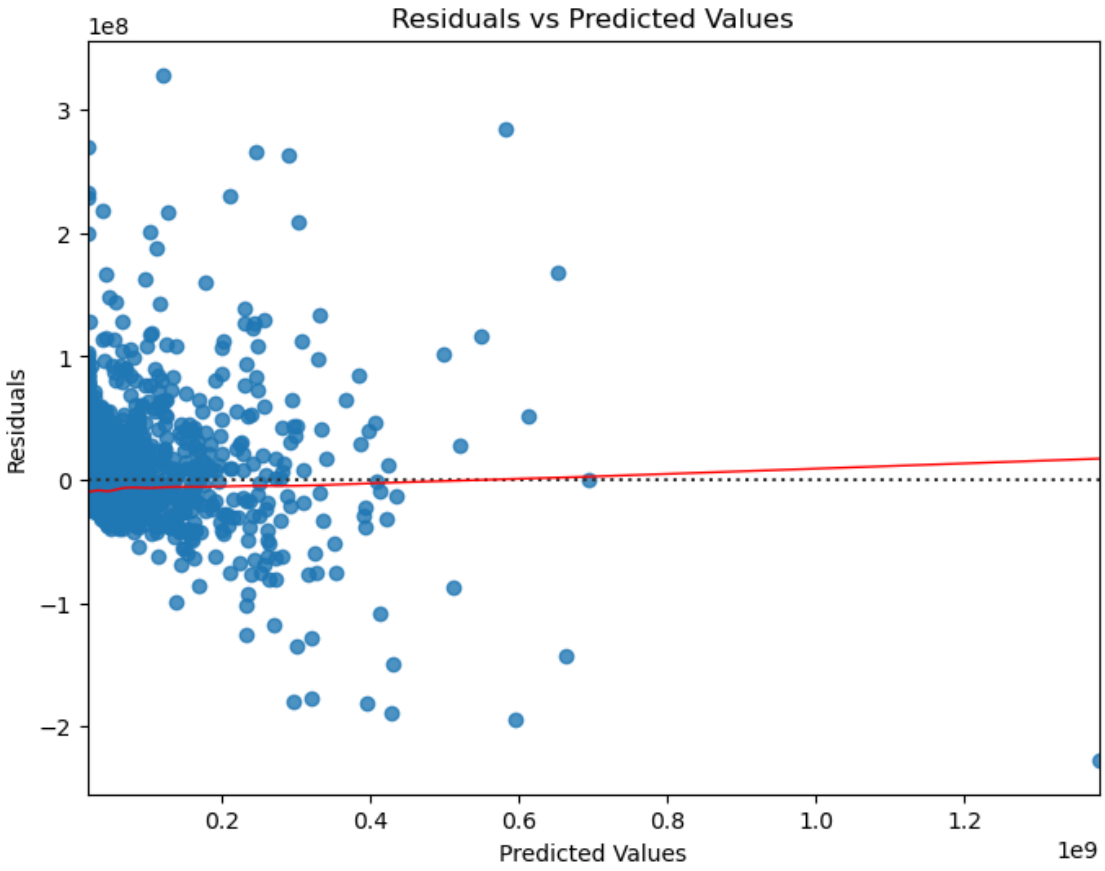
Test Set Score (R^2): The R^2 score of 0.76 indicates that my model explains about 76% of the variance in the target variable (total_xp). This is a solid performance for a regression task and suggests that the model is doing well, although there's still room for improvement.

4.1 Residual Plot

YOU HAVE BEEN WARNED. This is going to get ugly.

[102]:
```
# Residuals: Actual vs Predicted
residuals = y_test - y_pred

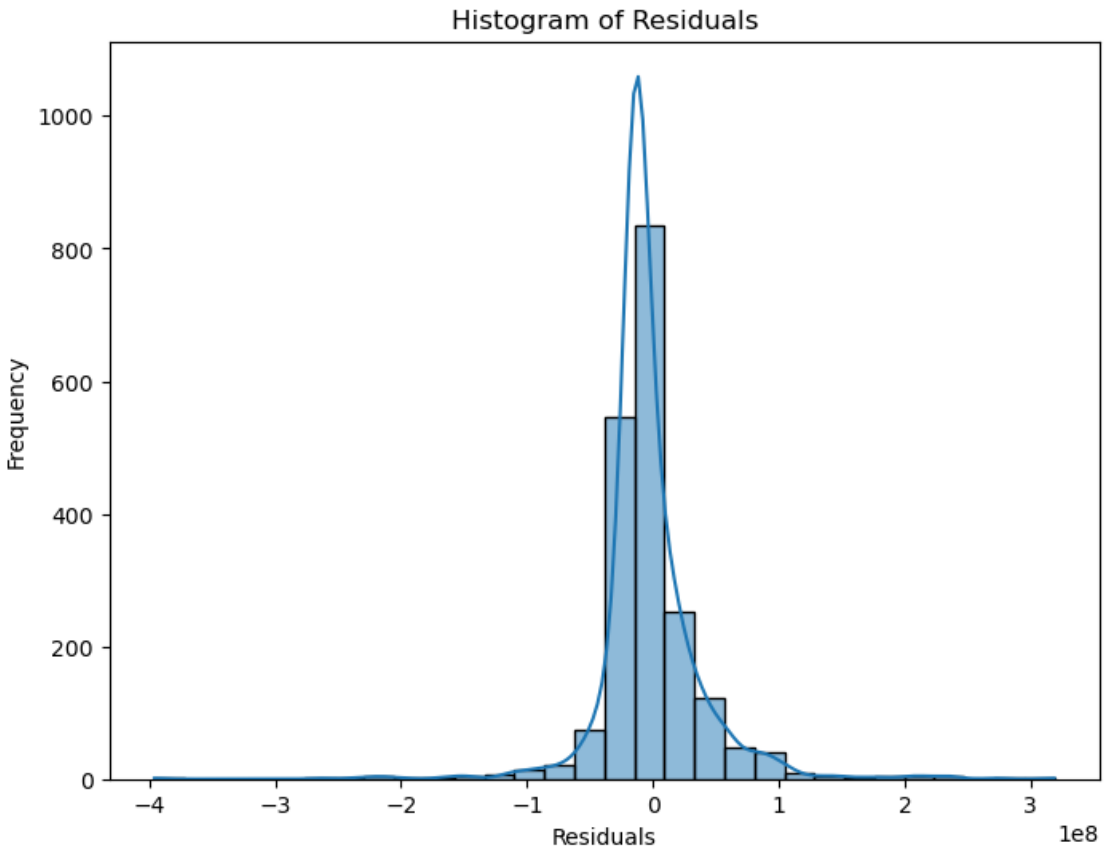
# 1. Residuals Plot
plt.figure(figsize=(8, 6))
sns.residplot(x=y_pred, y=residuals, lowess=True, line_kws={'color': 'red',␣
 ↪'lw': 1})
plt.title('Residuals vs Predicted Values')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.show()
```



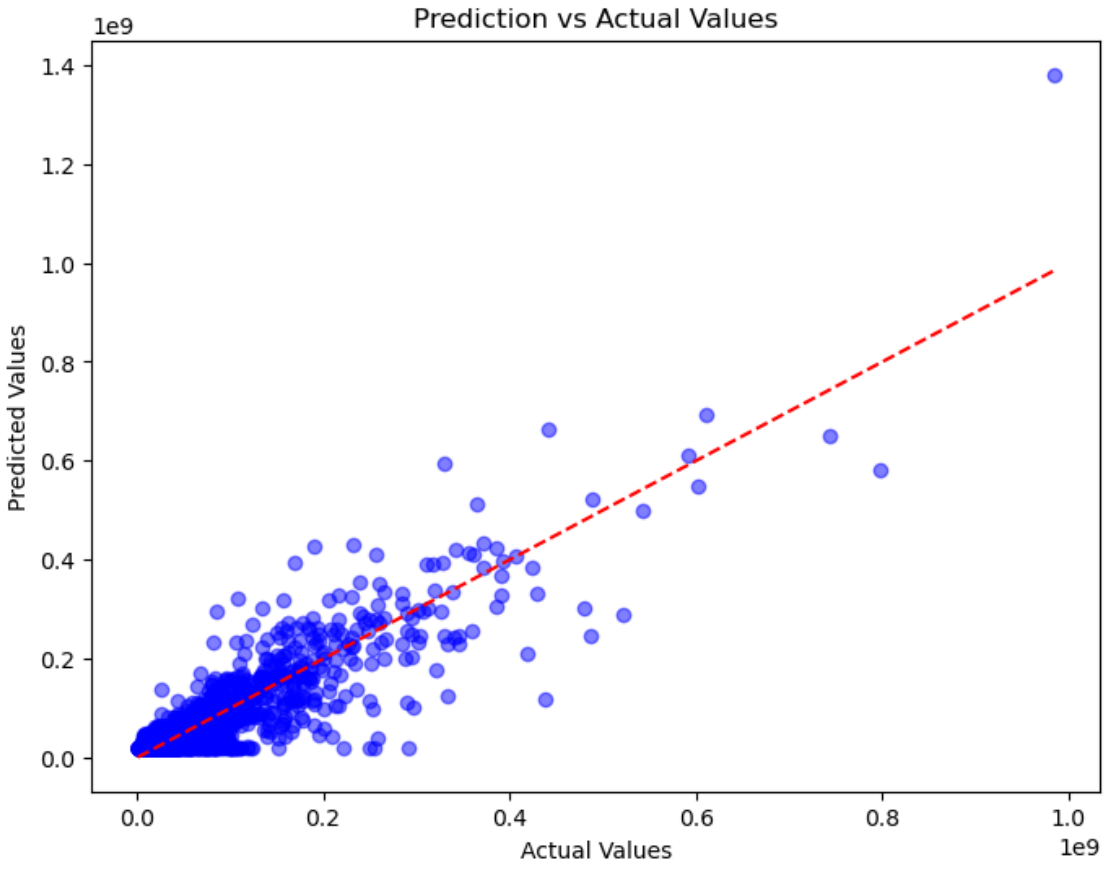
4.2 Histogram of Residuals

Very Leptokurtic!

[105]:
```
# 2. Histogram of Residuals
plt.figure(figsize=(8, 6))
sns.histplot(residuals, kde=True, bins=30)
plt.title('Histogram of Residuals')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.show()
```

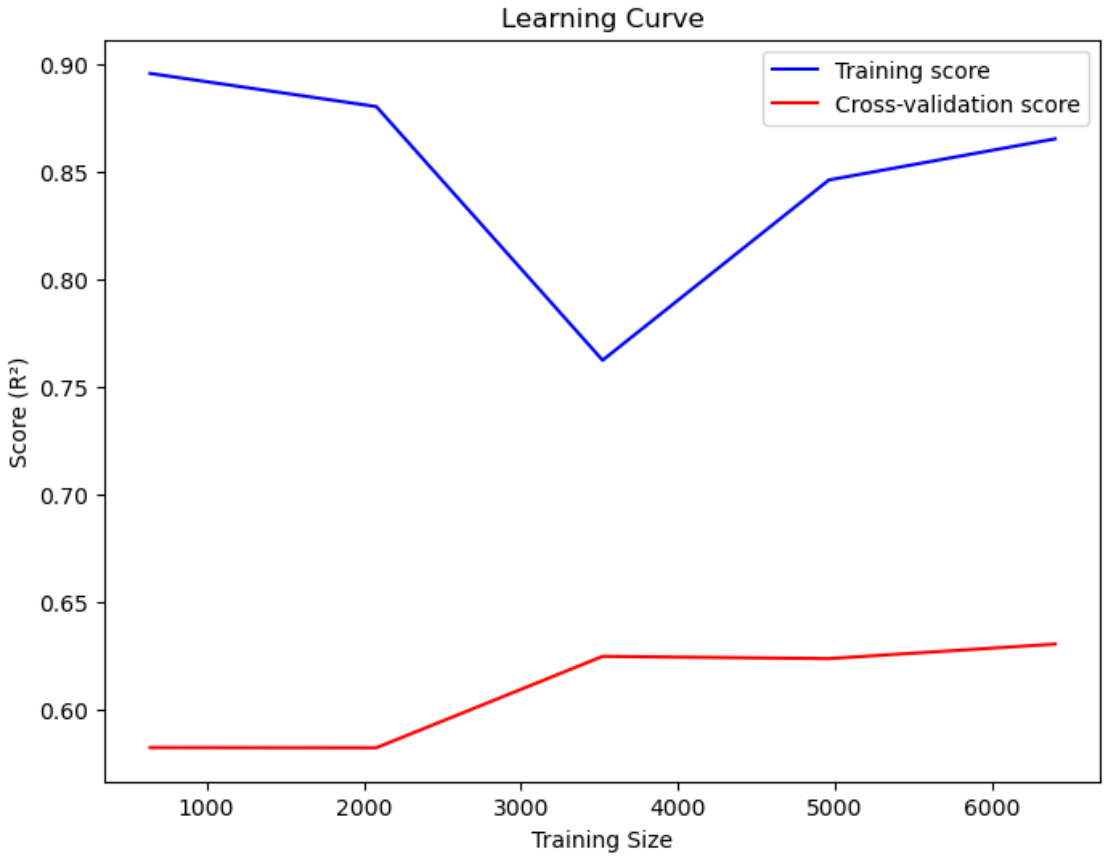


[109]:
```
# 3. Prediction vs Actual Plot
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],␣
 ↪color='red', linestyle='--')  # Line of perfect prediction
plt.title('Prediction vs Actual Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.show()
```



[113]:
```
# 4. Learning Curve
from sklearn.model_selection import learning_curve
train_sizes, train_scores, test_scores = learning_curve(best_rf_model, X_train,␣
 ↪y_train, cv=3, n_jobs=-1)

# Plot Learning Curve
plt.figure(figsize=(8, 6))
plt.plot(train_sizes, np.mean(train_scores, axis=1), label="Training score",␣
 ↪color="blue")
plt.plot(train_sizes, np.mean(test_scores, axis=1), label="Cross-validation␣
 ↪score", color="red")
plt.title('Learning Curve')
plt.xlabel('Training Size')
plt.ylabel('Score (R²)')
plt.legend(loc='best')
plt.show()
```

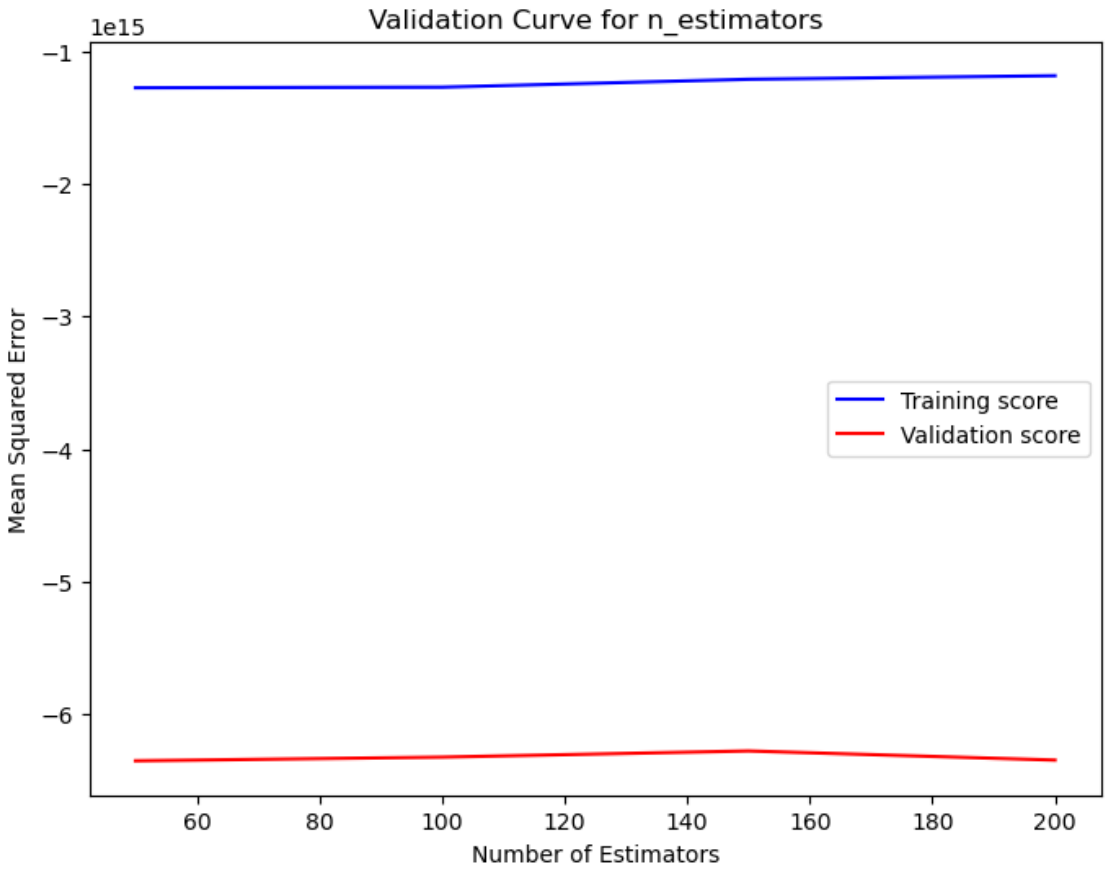


[115]:
```
# 5. Validation Curve for a Hyperparameter (e.g., n_estimators)
from sklearn.model_selection import validation_curve

param_range = np.arange(50, 201, 50)  # Varying the number of estimators in the␣
 ↪random forest
train_scores, test_scores =␣
 ↪validation_curve(RandomForestRegressor(random_state=42), X_train, y_train,
                                                 param_name="n_estimators",␣
 ↪param_range=param_range, cv=3, scoring="neg_mean_squared_error")

# Plot Validation Curve
plt.figure(figsize=(8, 6))
plt.plot(param_range, np.mean(train_scores, axis=1), label="Training score",␣
 ↪color="blue")
plt.plot(param_range, np.mean(test_scores, axis=1), label="Validation score",␣
 ↪color="red")
plt.title('Validation Curve for n_estimators')
plt.xlabel('Number of Estimators')
plt.ylabel('Neg Mean Squared Error')
```

```
plt.legend(loc='best')
plt.show()
```



5 Conclusion

1) Tuning the model could improve accuracy as well as including many possible models.

   The random forest could be tuned for all possible hyper parameters.

   I chose to only use the random forest to highlight its robustness to sparse data. To illustrate this I ahve included a simple linear regression

2) Grouping-based imputation and a cluster analysis

   The data could be 'smart' imputed, instead of eveerything getting the 0 treatment, although I have chosen to stick with the simple imputation, because it more closely resembles real life as most of these NAs are caused by players who have never been logged on the highscores in that metric, because they quite literally have 0 score for that activity.

3) Other Scoring Metrics

   Percent Mean Squared Error would be much more robust to outliers.

4) Log Transforms

   Applying a log transformation, instead of a standard scaler, after imputation would go a long way to aiding the extreme skew in every predictor

5) Feature Extraction and Engineering

   The 60 potential predictors could be engineered down to less than 10 columns by grouping the scores. Clue scrolls are already grouped with the 'Total Clue Scroll Score' column, but many others could be as well. Low-level, mid-level, and high-level bosses could be grouped together. The "skilling" bosses could also get this treatment as well.

[ ]: